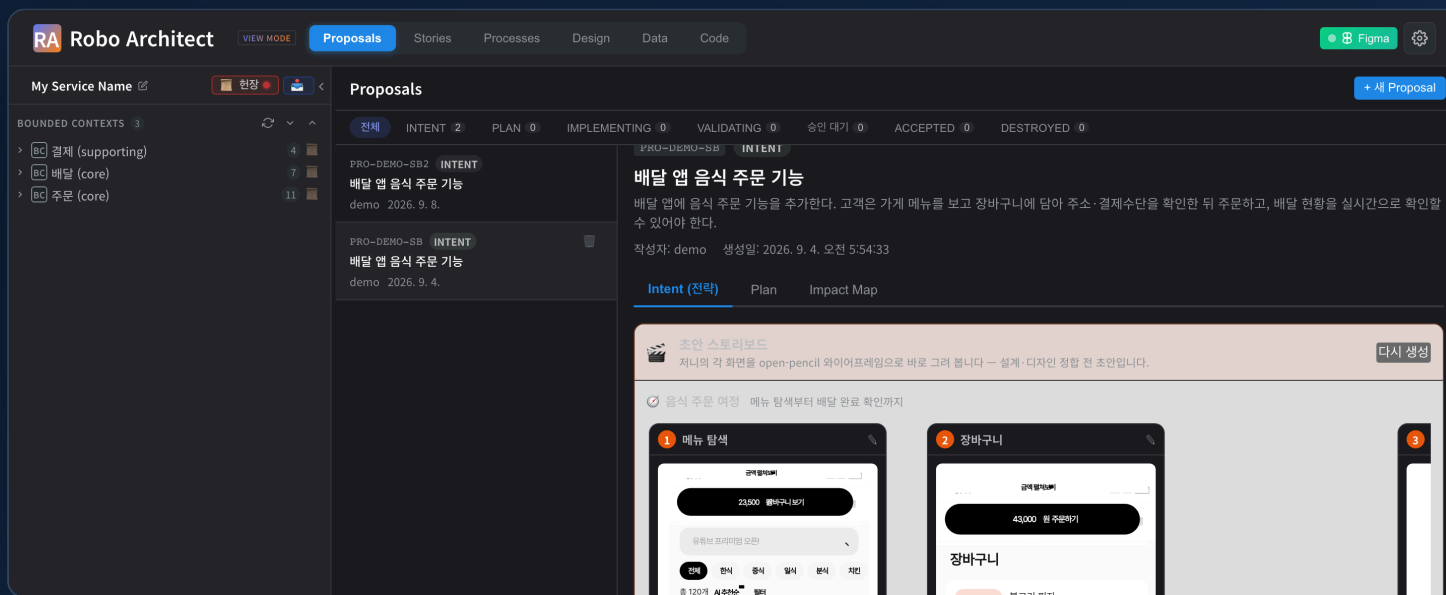


제 품 소 개 및 도 입 제 안 서

요구사항이 바뀌면, 설계도 함께 움직입니다.

요구사항과 업무 규칙을 도메인 그래프로 구조화하고, 변경의 영향 범위를 분석합니다.
설계부터 화면 구성과 코드 구현까지 연결하는 **AI 아키텍처 워크벤치**입니다.



요구사항 → 코드

한 흐름으로 연결

22종

도메인 그래프 노드

3가지 모드

간편 · DDD 상세 · ODA 표준

2026년 9월

AI 아키텍처 워크벤치 · 온프레미스 / 데스크톱 설치

uEngine

목차

01 AI 코딩의 역할	3	10 특징점 ④ 단계별 설계 지원	13
02 명세 · 시나리오 · 도메인	4	11 특징점 ⑤ 화면 초안 검토	14
03 여러 팀의 명세 관리	5	12 특징점 ⑥ 승인과 변경 반영	15
04 Robo Architect 소개	6	13 시스템 구성과 실행 환경	16
05 주요 산출물 ① 도메인 그래프	7	14 기술 스택	17
06 주요 산출물 ② 화면과 코드	8	15 사용 절차 (실제 화면)	18-19
07 제품 화면 한눈에 보기	9	16 보안 · 거버넌스 · 설치	20-21
08 특징점 ①② 그래프 · 영향 분석	10-11	17 개발 및 품질 검증 체계	22
09 특징점 ③ 제안부터 병합까지	12	18 MSAEZ 도입 실적 · 도입 절차	23-24

한 장 요약

AI를 활용한 개발에서는 **구현할 기능과 검증 기준을 명확히 정의하는** 일이 중요합니다. Robo Architect는 요구사항, 업무 시나리오, 도메인 모델을 하나의 그래프로 연결해 설계와 구현, 이후 변경까지 관리합니다.

SDD · BDD · DDD

명세로 요구사항을 정의하고, 시나리오로 검증 기준을 정하며, 도메인별로 설계 범위를 구분합니다

그래프가 기준

요구사항 · 설계 요소 · 화면 · 소스 파일의 연결 관계를 하나의 그래프로 관리합니다

설치형

고객사 환경에 설치하며, 폐쇄망에서는 사내 LLM 게이트웨이에 연결할 수 있습니다

설계와 구현을 연결하는 변경 관리

여러 팀이 각자의 저장소에서 명세를 관리하면 용어와 설계 기준이 달라지고, 변경의 영향 범위를 파악하기 어려워집니다. Robo Architect는 **명세와 설계 요소의 관계를 그래프로 관리**해 팀 간 정보를 연결하고, 요구사항에서 소스 파일까지 추적할 수 있도록 지원합니다.

이 문서가 다루는 것

제품 개요

3-15쪽
배경 · 정의 · 산출물 ·
특징점

시스템 구성

16-17쪽
시스템 구성 · 기술 스택

사용 절차

18-19쪽
실행 중인 제품 화면 ·
6단계

도입 안내

20-24쪽
보안 · 납품 · 실적 · 일정

AI 코딩의 역설

AI가 코드를 빠르게 작성하더라도 전체 개발 시간이 줄어드는 것은 아닙니다. 생성된 코드를 검토하고 수정하는 시간까지 고려해야 합니다. **요구사항과 업무 맥락, 검증 기준을 명확히 전달하는 것이 AI 활용의 출발점입니다.**

24% 단축

참여 개발자의 사전 예상

AI 사용 시 예상한 작업 시간 감소율

19% 증가

실제 작업 시간

AI 사용 시 작업 완료까지 걸린 시간

16명 · 246건

실험 참여자와 작업 수

숙련 개발자의 기존 오픈소스 프로젝트 작업

출처: [METR, 2025년 무작위 대조 실험](#). 당시 도구와 참여자의 작업 환경에서 측정한 결과입니다.

AI 활용 시 함께 관리해야 할 네 가지 문제

관리 항목 1

업무 맥락 전달

AI가 조직의 업무 규칙과 용어를 이해하려면 관련 정보를 제공해야 합니다. 이 정보가 정리되어 있지 않으면 담당자가 대화마다 다시 설명해야 하고, 누락된 내용은 잘못된 구현으로 이어질 수 있습니다.

관리 항목 2

검토와 수정 부담

코드가 빠르게 생성되어도 요구사항에 맞는지 확인하는 과정은 필요합니다. 검증 기준이 불명확하면 담당자의 판단에 의존하게 되고, 검토와 수정에 드는 시간이 늘어납니다.

관리 항목 3

모호한 요구사항

숙련 개발자는 업무 지식을 바탕으로 기획서의 누락된 내용을 확인합니다. AI에 같은 맥락이 제공되지 않으면 의도와 다른 결과가 나올 수 있으므로, 구현 전에 요구사항을 구체화해야 합니다.

관리 항목 4

설계 근거의 공유와 재사용

설계 결정과 그 근거가 대화 기록에만 남으면 다음 작업에서 찾고 활용하기 어렵습니다. 담당자가 바뀌어도 이어서 사용할 수 있도록 요구사항과 함께 관리해야 합니다.

AI가 작성할 코드의 요구사항과 검증 기준을 먼저 정해야 합니다. 업무 규칙과 설계 근거를 함께 관리하면, 구현과 검토에 필요한 맥락을 일관되게 제공할 수 있습니다.

명세 · 시나리오 · 도메인으로 맥락을 구체화합니다

Robo Architect는 세 가지 방법론을 연결합니다. **명세 주도 개발(SDD)**로 요구사항을 정의하고, **행동 주도 개발(BDD)**로 기대 동작을 구체화하며, **도메인 주도 설계(DDD)**로 업무 영역과 용어의 경계를 정합니다.

SDD · 명세 주도 개발

구현할 내용을 명세로 정의

목적과 요구사항을 문서로 정리한 뒤, 구현 계획과 작업 목록을 작성합니다. SI와 개발자가 같은 명세를 기준으로 작업할 수 있도록 합니다.

BDD · 행동 주도 개발

기대 동작을 사례로 설명

“잔액이 100원일 때 20원을 인출하면 잔액은 80원이 된다”
처럼 조건, 행동, 결과를 구체적으로 적습니다. 이 시나리오를 검증 기준으로 사용합니다.

DDD · 도메인 주도 설계

업무 영역별 설계 범위 구분

업무 개념과 규칙을 기준으로 도메인의 경계를 정합니다. 각 영역에서 사용하는 용어와 책임을 명확히 해 SI에 필요한 맥락을 전달합니다.

현금 인출 요구사항을 구체화하는 예

작성 방식	예시 — ATM 현금 인출	검토 시 확인할 점
사용자 스토리	“고객은 계좌에서 현금을 인출할 수 있어야 한다”	잔액 부족 등 예외 상황과 처리 조건을 추가해야 합니다
요구사항 명세서	“시스템은 인증된 사용자에게 현금 인출 기능을 제공한다”	인증 조건과 인출 한도, 실패 시 동작을 구체화해야 합니다
상세 설계서	<pre>if balance >= amount: balance -= amount</pre>	처리 로직을 표현하지만, 비개발자가 검토하기는 어렵습니다
BDD 시나리오	Given 잔액 100원 · When 20원 인출 · Then 현금 20원 지급, 잔액 80원	업무 담당자와 개발자가 같은 사례로 기대 결과를 확인합니다

BDD 시나리오는 업무 규칙을 함께 검토하는 기준입니다. 기획자와 고객은 조건과 결과가 업무 의도에 맞는지 확인하고, AI는 이를 테스트 작성의 근거로 활용합니다. Robo Architect는 시나리오를 관련 설계 요소와 연결해 그래프에 저장합니다.

여러 팀의 명세를 연결해 관리해야 합니다

파일 기반 명세는 팀의 요구사항과 구현 계획을 정리하는 데 유용합니다. 다만 **여러 팀과 저장소에 정보가 분산되면** 용어를 맞추고 변경의 영향 범위를 파악하기 어려워집니다. 조직 차원에서는 다음 네 가지 문제를 함께 관리해야 합니다.

관리 과제	발생 원인	업무에 미치는 영향
분산된 명세	명세 파일이 각 저장소 안에만 존재합니다	다른 팀의 구현을 파악하기 어려워 같은 기능을 중복 개발할 수 있습니다
용어 불일치	저장소마다 도메인 용어를 각자 정의합니다	“수용가”와 “고객”처럼 서로 다른 용어가 같은 대상을 뜻하는지 확인해야 합니다
영향 범위 파악	요구사항 · 설계 · 코드의 관계를 문서 링크로 관리합니다	규정이 바뀌면 관련 문서와 코드를 일일이 확인해야 합니다
설계 지식의 인계	업무 지식과 설계 근거가 담당자와 대화 기록에 의존합니다	담당자 변경 시 업무 지식이 누락되거나 같은 분석을 반복할 수 있습니다

Robo Architect는 **요구사항 · 도메인 모델 · 화면 · 소스 파일의 관계**를 하나의 그래프로 관리합니다. 변경 요청이 들어오면 연결된 설계 요소와 구현 파일을 따라 영향 범위를 확인할 수 있습니다.

파일 기반과 그래프 기반의 차이

비교 항목	파일 기반 명세	그래프 기반 명세
명세의 위치	저장소 안의 마크다운 파일	여러 팀이 공유하는 도메인 그래프
경계의 기준	저장소와 폴더 단위	Bounded Context 단위의 업무 영역
용어 일관성	저장소마다 각자 기술	도메인별 용어와 관계를 함께 관리
변경 영향 분석	파일 검색과 담당자 협의로 확인	연결 관계를 따라 영향 범위와 충돌 가능성 분석
지식의 축적	프로젝트별 파일에 기록	설계 요소와 변경 이력으로 연결해 보존

파일 기반 도구와 그래프 기반 관리는 함께 활용할 수 있습니다. 팀은 기존 저장소에서 명세와 코드를 관리하고, Robo Architect는 설계 요소 간 관계와 변경 이력을 연결해 관리합니다.

Robo Architect 소개

요구사항 분석부터 설계, 화면 구성, 코드 구현까지의 과정을 하나의 도메인 그래프로 연결하는 **AI 아키텍처 워크벤치**입니다. 요구사항 문서를 업로드하면 이벤트스토밍 기반의 설계 그래프를 생성합니다. 변경을 제안하면 영향 범위와 구현 계획을 제시하고, 승인된 변경 사항을 코드와 설계 그래프에 함께 반영합니다.

입력

요구사항

기획 문서(PDF · 텍스트)를 업로드하거나 Confluence · Figma 자료를 연결합니다. 구현할 기능을 글로 직접 입력할 수도 있습니다.

처리

도메인 그래프로 구조화

요구사항을 Bounded Context, Aggregate, 명령, 이벤트 등으로 구조화하고, 관련 화면과 함께 그래프에 저장합니다.

출력

설계 · 화면 · 코드

설계 그래프, 영향 분석 결과, 구현 계획, 화면 초안과 소스 코드를 생성합니다.

기존 방식과의 차이

	문서 중심의 설계 관리	Robo Architect
설계의 형태	다이어그램 이미지와 문서	질의 가능한 그래프(노드 22종 · 관계 32종)
변경 영향	담당자가 문서와 코드를 대조	그래프 탐색으로 영향받는 설계 요소 식별
화면	별도 도구에서 따로 관리	같은 그래프의 UI 노드 — 초안 단계부터 생성
코드	설계 문서와 별도로 관리	구현 파일과 설계 요소의 연결 관계를 기록
방법론	교육 자료	단계별 절차로 제품에 내장(DDD 6단계 · ODA 표준)
AI의 역할	개별 작업 지원	요구사항 구체화부터 구현 · 검증까지 단계별 승인 절차 지원

한눈에 보는 흐름



사용자가 단계별 결과를 확정하면 다음 단계로 진행하며, **승인된 변경 사항만** 반영됩니다.

주요 산출물 ① — 도메인 그래프

요구사항 문서를 업로드하면 **21단계 처리 과정**을 거쳐 요구사항에서 도메인 개념과 관계를 추출하고 그래프에 저장합니다. 진행 상황은 단계별로 표시되며, 중간에 멈춰 검토한 뒤 이어서 진행할 수 있습니다.



그래프에 저장하는 정보

계층	노드
요구사항	Requirement · UserStory · Feature · Given/When/Then(인수 조건)
전략 설계	BoundedContext · Policy
전술 설계	Aggregate · Command · Event · ReadModel · Property · Invariant
화면 · 흐름	UI · Journey · JourneyStep · FigmaBinding · StoryboardPageMapping
구현	ImplementationFile — 설계 요소를 구현한 소스 파일과의 연결

22종

노드 유형

요구사항부터 구현 파일까지

32종

관계 유형

EMITS · TRIGGERS · IMPLEMENTS · EFFECT ...

21단계

문서 수집 · 분석 과정

단계별 진행 표시 · 중단/재개

설계 요소의 관계를 따라 필요한 정보를 찾습니다. “이 이벤트를 발생시키는 명령은 무엇인가?”, “이 요구사항을 구현한 파일은 무엇인가?”, “이 Aggregate를 수정하면 어떤 화면이 영향을 받는가?”처럼 설계와 구현을 함께 조회할 수 있습니다.

설계 정보의 단일 기준

설계 정보는 그래프를 기준으로 관리합니다. 화면과 문서를 별도의 기준 데이터로 유지하는 대신, **그래프의 현재 상태에서 다시 생성**할 수 있도록 구성했습니다.

주요 산출물 ② — 화면과 코드

도메인 그래프를 바탕으로 **화면 초안과 코드**를 생성합니다. 생성된 소스 파일은 해당 설계 요소와 연결해 기록하므로, 이후 변경 시 구현 위치를 추적할 수 있습니다.

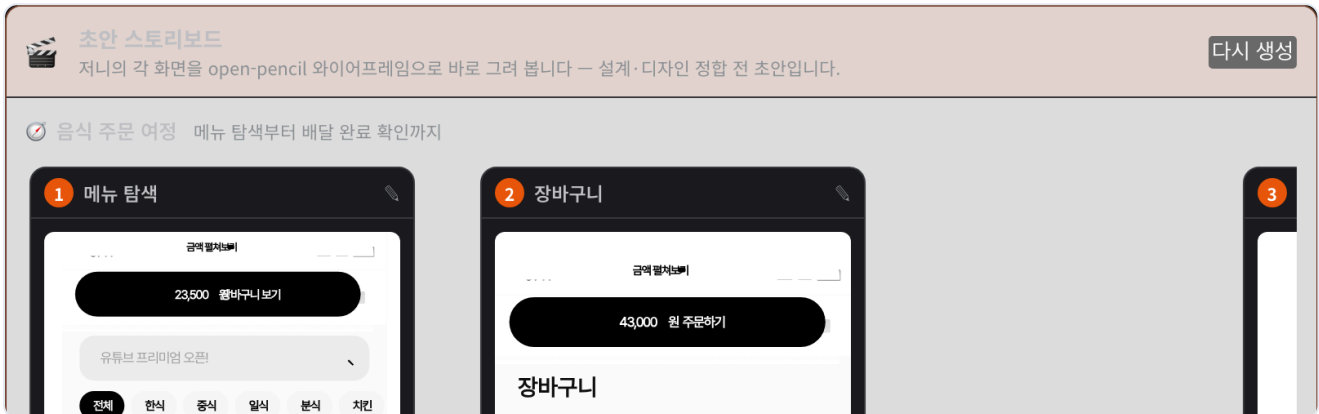


그림 1. 자동 생성된 스토리보드. 화면 초안을 사용 순서에 따라 배치하고, 조건에 따른 분기를 마름모로 표시합니다. *데모 데이터를 사용한 화면입니다.*

직접 편집할 수 있는 화면 초안

와이어프레임은 개별 요소를 수정할 수 있는 구조로 생성됩니다. 내장 편집기에서 수정하거나 Figma로 내보내 디자인 작업을 이어갈 수 있습니다. 사내 디자인 시스템의 컴포넌트도 활용할 수 있습니다.

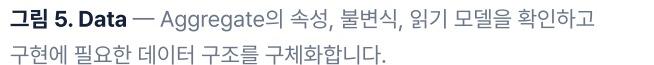
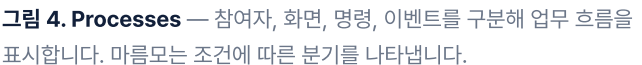
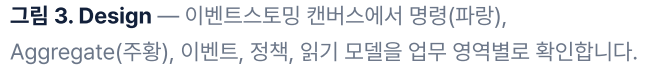
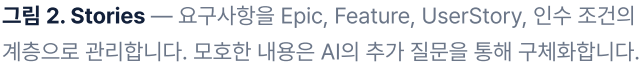
별도 작업 공간에서 코드 구현

대상 프로젝트의 **격리된 작업 공간**(git worktree)에서 코드를 작성한 뒤, 승인하면 대상 브랜치에 병합합니다. 구현 도중 작업을 중단하고 수정 지시를 전달한 뒤 다시 진행할 수 있습니다.

산출물 내보내기

산출물	내용
PRD · 기능 명세	Bounded Context 단위로 요구사항·설계·인수 조건을 묶어 생성
이벤트 모델	UI → 명령 → 이벤트 → 읽기 모델 스퀴레인, GWT 시나리오 포함
BPMN 프로세스	업무 흐름을 표준 BPMN으로 내보내기
화면 · 스토리보드	와이어프레임, Figma 프레임, <code>.fig</code> 파일
사용자 매뉴얼	실제 화면을 캡처해 DOCX 매뉴얼로 자동 생성

최신 설계를 기준으로 산출물을 다시 생성할 수 있습니다. 그래프의 변경 사항을 반영해 문서를 생성하므로, 설계와 문서 간 불일치를 줄일 수 있습니다.



Robo Architect 제품 소개서

① 설계 요소와 관계를 그래프로 관리합니다

Robo Architect는 **설계 요소와 그 관계를 그래프에 저장**합니다. 다이어그램은 이 정보를 시각화한 화면입니다. 같은 정보를 역할에 맞게 조회하고, 문서와 화면으로 다시 생성할 수 있습니다.

그래프로 연결해 활용하는 설계 정보

- **조회** — 특정 이벤트에 반응하는 정책이나 읽기 모델을 사용하는 화면을 찾습니다.
- **추적** — 요구사항과 설계 요소의 연결 관계를 따라 해당 기능을 구현한 소스 파일을 확인합니다.
- **재생성** — 최신 그래프를 기준으로 문서, 다이어그램, 화면을 다시 생성합니다.
- **협업** — 같은 그래프를 기획자는 요구사항 트리로, 아키텍트는 이벤트 모델로, 디자이너는 화면 흐름으로 봅니다.

역할별 화면 구성

기획자 · PO

요구사항 트리

요구사항을 Epic, Feature, UserStory, 인수 조건으로 구분해 관리합니다. 모호한 내용은 AI의 추가 질문에 대해 구체화합니다.

아키텍트

이벤트스토밍 캔버스

Bounded Context 경계, Aggregate, 명령 · 이벤트 · 정책을 한 화면에서 봅니다. 노드를 선택해 속성과 불변식을 확인하고 수정할 수 있습니다.

개발자

Aggregate · CQRS 뷰

Aggregate의 속성과 불변식, 읽기 모델을 검토해 구현에 필요한 데이터 구조를 구체화합니다.

디자이너

화면 흐름 · Figma 연동

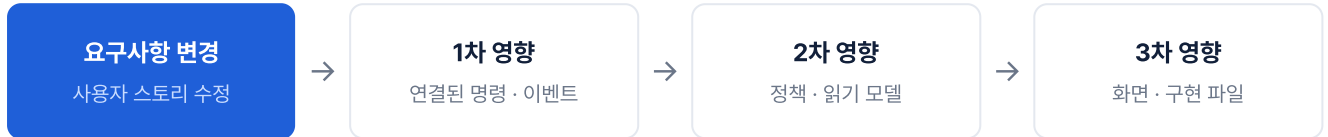
사용자 흐름과 화면을 확인하고, Figma 문서와 양방향으로 동기화합니다.

업무 의미가 드러나는 용어를 사용합니다

화면, API, 데이터 모델에서 같은 도메인 용어를 사용합니다. 단순한 생성 · 수정 · 삭제 대신 **“주문 취소” 명령, “주문 취소됨” 이벤트**처럼 업무 의미를 표현해 요구사항과 설계, 구현을 연결합니다.

② 변경에 따른 영향 범위를 자동으로 분석합니다

요구사항을 변경할 때 **연결된 설계 요소와 소스 파일에 미치는 영향**을 분석합니다. 그래프의 관계를 따라 검토가 필요한 화면, 업무 규칙, 구현 파일을 제시합니다.



영향 분석 결과

항목	내용
영향 요소 목록	변경의 영향을 받는 설계 요소를 계층별로 제시하고, 확신도(높음 · 중간 · 낮음)와 판단 근거를 함께 표시합니다
충돌 가능성	동일한 설계 요소에 영향을 주는 다른 변경 요청이 있으면 함께 표시합니다
회귀 테스트 범위	영향 경로에 포함된 인수 조건(GWT)을 모아 재검증 대상으로 제시합니다

원본 설계를 변경하지 않고 적용 후의 구조를 미리 확인합니다. 기존 그래프 화면에서 변경안을 검토할 수 있으며, 검토를 중단해도 원본 설계는 유지됩니다.

분석 예시: 주문 취소 시 환불 규칙 변경

“주문 취소 시 환불 규칙을 변경해 주세요”라는 요청의 분석 예시입니다.

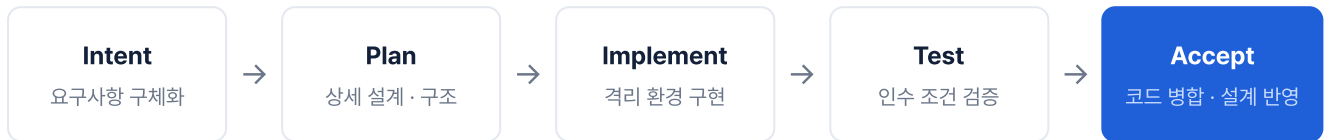
계층	영향받는 설계 요소
요구사항	주문 취소 · 환불 확인 관련 UserStory와 인수 조건
전술 설계	주문 취소 명령, 주문 취소됨 이벤트, 환불 정책, 결제 불변식
화면	주문 취소 화면, 주문 현황 화면
구현	해당 설계 요소에 연결된 소스 파일

변경 이력

변경 요청마다 고유 번호를 부여하고 관련 설계 요소와 연결합니다. 요청은 초안 → 제출 → 승인 → 구현 완료 순서로 진행되며, 요청자는 본인이 제출한 변경을 승인할 수 없습니다. 이력에서 설계 변경 시점과 근거를 확인할 수 있습니다.

③ 변경 제안부터 구현과 병합까지 연결합니다

추가하거나 변경할 기능을 글로 입력하면 AI가 요구사항을 세분화하고 구현 계획을 제시합니다. 사용자는 단계별 결과를 검토하고, 승인한 계획에 따라 구현과 검증을 진행합니다. 최종 승인 후에는 구현 코드를 병합하고 설계 그래프에도 변경 사항을 반영합니다.



단계	하는 일
Intent	입력한 글을 Epic · Feature · UserStory · 사용자 흐름으로 구체화합니다. 내용이 모호하면 최대 5개의 선택형 질문으로 확인합니다
Plan	확정된 요구사항과 프로젝트 설계 원칙 (기술 스택 · 모놀리스/마이크로서비스 · 저장소 구성 전략)을 근거로 Aggregate · 명령 · 이벤트와 배포 · 연동 계획을 산출합니다
Implement	대상 프로젝트에 전용 브랜치의 격리된 작업 공간을 만들고, Code 탭에서 Claude Code가 구현을 수행합니다. 중지 · 수정 지시 · 재시도가 가능합니다
Test	인수 조건(GWT)을 기준으로 결과를 검증하고 구조 검사를 함께 수행합니다
Accept	코드 병합과 설계 그래프 반영을 함께 수행합니다. 한쪽이 실패하면 양쪽을 되돌립니다

코드와 설계를 함께 반영합니다

승인하면 코드 병합과 설계 그래프 반영이 함께 진행됩니다. 어느 한쪽이 실패하면 양쪽을 되돌리고 실패 상태로 표시해 재시도할 수 있습니다. **코드만 병합되고 설계에는 변경 사항이 반영되지 않는 상황**을 방지하기 위한 구조입니다.

승인을 취소하면 설계 그래프에 반영한 변경을 되돌리며, 코드 되돌림 여부는 선택할 수 있습니다. 제안을 폐기하면 격리된 작업 공간도 함께 정리됩니다.

④ 단계별 질문으로 설계 결정을 지원합니다

DDD를 실무에 적용할 수 있도록 **단계별 절차와 검토 질문**을 제공합니다. 제안 작성 시 요구사항을 구체화할 방식인 분해 모드를 선택하면, 변경 범위에 맞는 설계 절차를 안내합니다.

세 가지 분해 모드

간편(Simplified)

요구사항 구체화와 계획 두 단계로 진행합니다. 영향 범위가 작고 신속한 처리가 필요한 변경에 적합합니다.

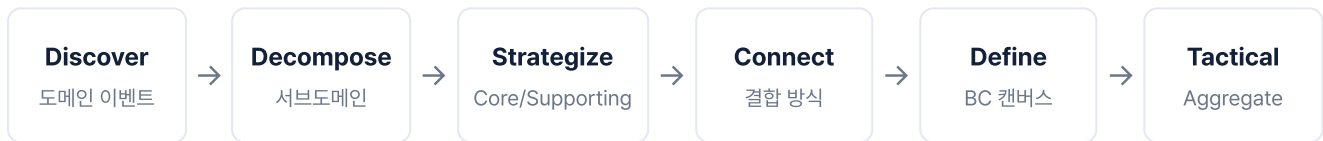
DDD 상세(Detailed)

ddd-starter 방법론에 따라 6단계로 설계합니다. 새로운 업무 영역을 설계하거나 시스템 구조를 크게 변경할 때 적합합니다.

ODA 표준

통신 업무 영역을 위한 모드입니다. TM Forum의 ODA · SID · Open API 표준을 기준으로 설계하고 적합성을 점검합니다.

DDD 상세 모드의 6단계



단계마다 **설계 결정에 필요한 질문**을 제시합니다. "이 서브도메인이 사업의 핵심 경쟁력을 담당하는가?", "서비스 간 연동에 이벤트 방식을 사용할 것인가, 동기 호출을 사용할 것인가?", "이 Aggregate가 지켜야 할 불변식은 무엇인가?" 같은 질문에 선택형으로 답하고, 필요하면 이전 단계로 돌아가 다시 생성합니다. 먼저 **변경 범위를 분석**해 필요한 단계를 제안하므로, 작은 변경에 6단계를 모두 적용하지 않습니다.

공통 설계 결정을 다음 제안에도 활용합니다. 도메인의 Core/Supporting/Generic 분류와 기본 연동 방식 등은 프로젝트 설계 원칙으로 저장합니다. 이후 제안에서도 같은 기준을 적용하므로 담당자가 바뀌어도 설계 근거를 이어서 활용할 수 있습니다.

절차는 스킴 파일로 관리합니다

각 단계는 독립된 **스킬**(AI의 작업 절차와 규칙을 정의한 파일, 총 18종)로 구현되어 있습니다. 방법론이 바뀌거나 조직 고유의 절차를 추가할 때는 스킴만 수정하면 되며, 제품 코드를 고칠 필요가 없습니다.

⑤ 초안 단계에서 화면을 확인하고 수정합니다

요구사항을 글로만 검토하면 실제 화면과 사용 흐름을 파악하기 어렵습니다. Robo Architect는 **요구사항을 기능과 사용자 흐름으로 구체화한 뒤** 화면 초안을 자동으로 생성합니다.

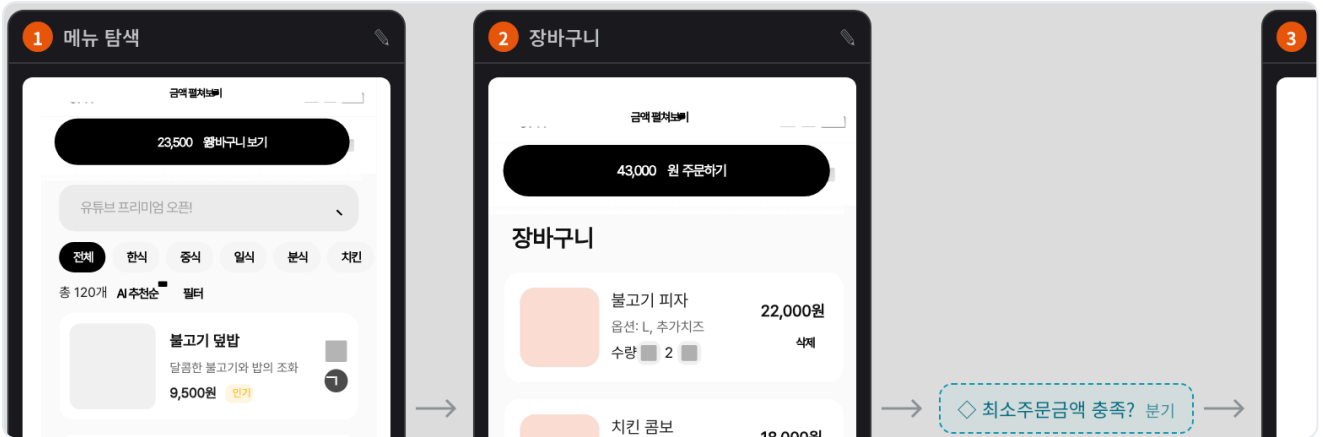


그림 6. 화면 초안을 사용 순서에 따라 배치한 스토리보드입니다. 조건에 따른 분기는 마름모로 표시하며, 각 화면은 개별 요소를 편집할 수 있습니다. 데모 데이터를 사용한 화면입니다.

초안 단계에서 화면 확인

상세 설계와 디자인을 마치기 전에도 주요 화면과 이동 흐름을 확인할 수 있습니다. 화면 초안을 보며 요구사항을 검토하고 보완합니다.

화면에서 바로 수정

카드의 편집 아이콘을 누르면 디자인 편집기가 열립니다. 요소를 옮기거나 크기를 바꾼 뒤 저장하면 해당 단계의 화면에 수정 사항이 반영됩니다.

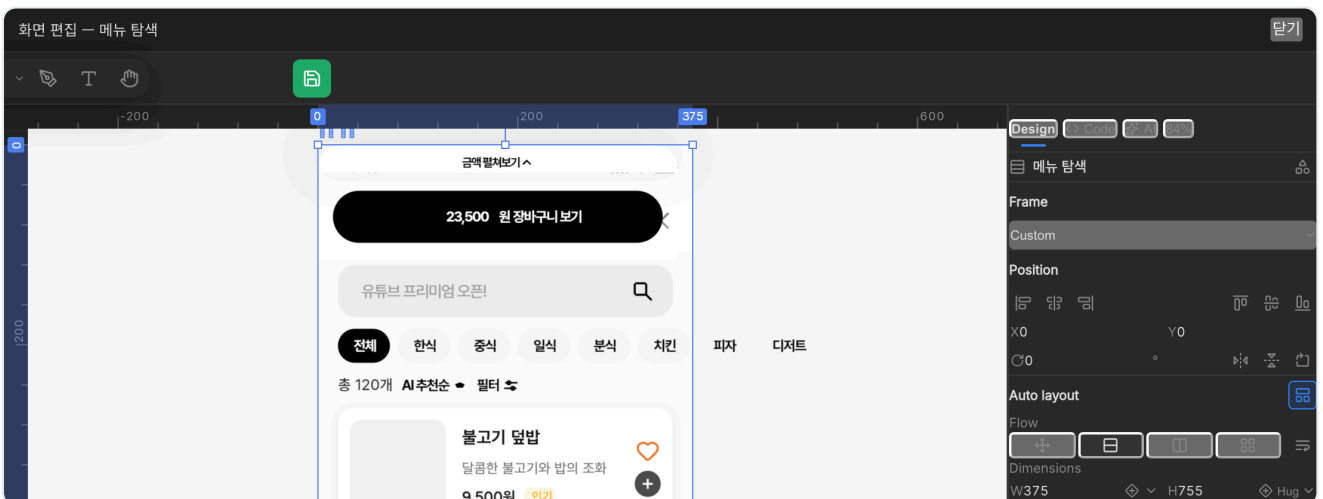


그림 7. 내장 디자인 편집기. 캔버스, 도구 모음, 속성 패널에서 요소의 위치, 정렬, 자동 배치, 색상을 조정합니다.

사내 디자인 시스템을 적용할 수 있습니다. Figma 컴포넌트 라이브러리를 연결하면 기존 버튼·헤더·카드 등을 활용해 화면을 구성합니다. 컴포넌트의 유형과 문구를 화면에 맞게 설정하고, 내장 편집기에서 직접 수정할 수 있습니다.

⑥ 최종 변경 사항은 승인 후 반영됩니다

사용자는 AI가 제안한 변경이 **언제, 어디에 반영되는지** 미리 확인할 수 있어야 합니다. Robo Architect는 모든 변경을 **제안** → **확인** → **적용** 세 단계로 나눕니다.



- 분석과 미리보기는 원본 설계를 변경하지 않습니다. 영향 범위와 화면 초안을 먼저 확인할 수 있습니다.
- 수정안 검토와 적용을 분리합니다. 추가 질문에 답하면 AI가 요구사항 수정안을 제시하고, 사용자가 승인한 뒤 적용합니다.
- 동시 편집으로 인한 덮어쓰기를 방지합니다. 다른 사람이 먼저 수정했다면 충돌로 알리고 덮어쓰지 않습니다.
- 되돌릴 수 있습니다. 변경마다 이력이 남으며, 승인을 취소하면 설계 그래프에 반영한 변경을 되돌리고 코드 되돌림 여부는 선택합니다.

작업별 최종 반영 시점

작업	원본에 최종 반영	설명
영향 분석 · 미리보기 · 스토리보드	없음	원본 설계와 대상 브랜치는 변경하지 않습니다
제안 내부의 초안 편집	없음	초안은 제안에 저장하고 원본 설계는 유지합니다
격리 환경에서의 구현	없음	격리된 작업 공간의 파일만 바뀌며, 대상 브랜치는 그대로입니다
계획(Plan) 확정	없음	구현 계획만 확정합니다
최종 승인(Accept)	있음	코드 병합과 그래프 반영이 함께 일어납니다

작업 진행 상황과 근거 확인

문서 수집 · 분석, 요구사항 구체화, 코드 구현, 화면 생성 등 시간이 걸리는 작업의 진행 상황을 실시간으로 표시합니다. **수행 중인 작업과 그 이유**를 한국어로 기록해 처리 과정과 결과의 근거를 확인할 수 있습니다.

요청별 처리 이력 추적

각 요청에 추적 번호(상관관계 ID)를 부여하고 처리 내용을 구조화된 로그로 기록합니다. 문제가 발생하면 같은 번호의 기록을 따라 실패한 단계와 원인을 확인할 수 있습니다.

시스템 구성과 실행 환경

데스크톱 애플리케이션 하나에 **웹 UI · API 서버 · 그래프 데이터베이스 · 렌더링 엔진**이 함께 들어 있습니다. LLM 요청의 전송 대상은 설정에서 지정하며, 사내 게이트웨이로 연결할 수 있습니다.



요청 처리 흐름

- 1 **요청 접수** — 사용자가 문서를 업로드하거나 제안을 작성하면 API 서버가 요청을 받습니다.
- 2 **작업 실행** — 서버가 해당 단계의 스킬을 코딩 에이전트로 실행하고, 진행 상황을 화면에 전달합니다.
- 3 **설계 정보 저장** — 요구사항 분석 결과와 설계 요소를 Neo4j 그래프에 저장합니다.
- 4 **화면 생성** — 렌더링 엔진이 JSX를 바탕으로 배치를 계산하고 편집 가능한 화면 구조를 생성합니다.
- 5 **코드 구현** — 대상 프로젝트의 별도 작업 공간에서 구현한 뒤, 승인하면 대상 브랜치에 병합합니다.

기술 스택

고객사 개발팀이 구조를 이해하고 유지보수할 수 있도록 **널리 사용되는 프레임워크와 오픈소스 기술**을 중심으로 구성했습니다.

레이어	구성
프론트엔드	Vue 3.5 · Vite · Pinia · Vue Flow(그래프 캔버스) · bpmn-js · Mermaid · Tailwind CSS · CodeMirror 6 · xterm.js
백엔드	Python 3.11+ · FastAPI · Uvicorn · Pydantic v2 · SSE(서버 전송 이벤트)
그래프	Neo4j 5 · Cypher · APOC. 스키마(제약·인덱스)는 파일로 형상관리
AI 오케스트레이션	LangChain · LangGraph · Deep Agents — 공급자에 종속되지 않는 인터페이스
LLM 공급자	Anthropic · OpenAI · Google · OpenRouter · OpenAI 호환 사내 게이트웨이
디자인 렌더링	open-pencil 엔진 · CanvasKit(Skia) · Yoga 레이아웃 · Figma <code>.fig</code> 입출력
코딩 에이전트	Claude Code(PTY 세션) · MCP 서버 · git worktree 샌드박스
데스크톱	Electron 31 · electron-builder(dmg · nsis · Applmage) · 자동 업데이트
컨테이너	Podman(rootless · 데몬리스) 또는 Docker · Compose 호환
검증	pytest · Playwright · ruff — 정적 분석 · 단위 · E2E

세 가지 설계 원칙

공급자 종립

LLM 공급자와 모델은 환경 설정으로 바꿉니다. 특정 공급자에 종속되지 않도록 연동을 구성했습니다.

기능별 모듈

백엔드와 프론트엔드를 같은 기능 단위로 구성합니다. 기능 모듈 간 직접 참조를 제한해 각 기능을 독립적으로 관리합니다.

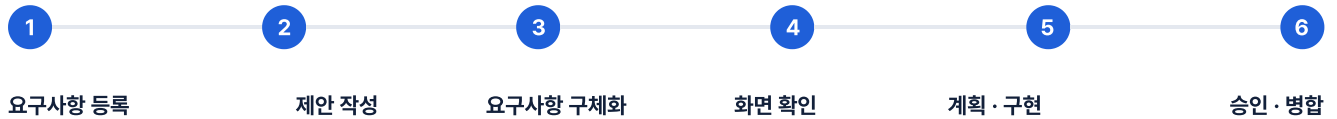
스킬 우선

방법론과 절차는 스킬 파일로 관리합니다. 절차를 바꾸는 데 제품 재배포가 필요 없습니다.

화면 생성에는 오픈소스 open-pencil 엔진을 사용합니다. 원본 프로젝트의 변경 사항을 정기적으로 반영하며, Figma `.fig` 파일을 가져오고 내보낼 수 있습니다.

사용 절차 — 여섯 단계

요구사항을 등록하고 변경을 제안한 뒤, 설계와 화면을 검토하고 구현 · 검증 · 병합을 진행합니다. 아래는 **실제 제품 화면으로 살펴보는 여섯 단계의 사용 절차**입니다.



1단계 · 요구사항 등록

기획 문서를 업로드하거나 Confluence 페이지를 연결하면 21단계 처리 과정을 거쳐 요구사항과 설계 요소를 추출해 그래프에 저장합니다. 진행 상황을 확인하면서 중단하거나 재개할 수 있습니다. 기존 설계 데이터가 있으면 새로 분석하는 대신 가져와 사용할 수도 있습니다.

2단계 · 변경 제안 작성

Proposals 탭에서 새 제안을 만들고 추가하거나 변경할 기능을 글로 입력합니다. 변경 범위와 설계 방식에 맞춰 **분해 모드** (간편 · DDD 상세 · ODA 표준)를 선택합니다.

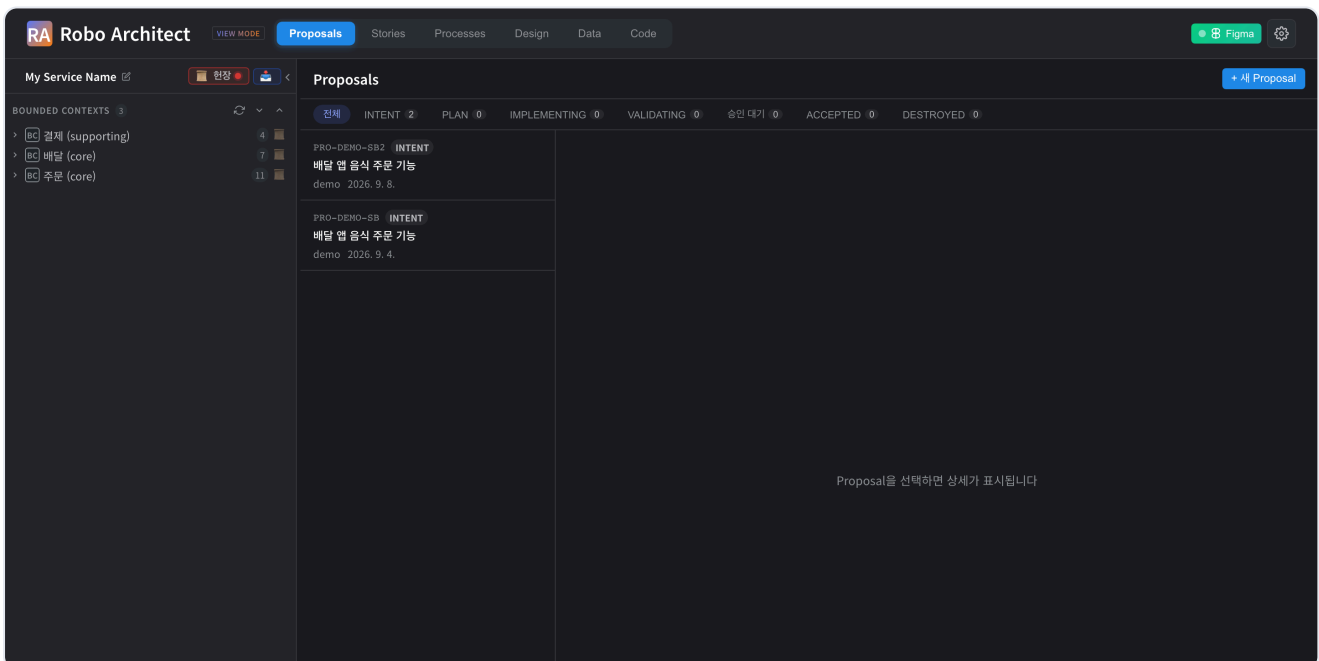


그림 8. 제안 목록 — 상태별 필터(Intent · Plan · Implementing · Validating · 승인 대기 · Accepted · Destroyed)로 진행 중인 변경을 한눈에 봅니다. *데모 데이터를 사용한 화면입니다.*

3단계 · 요구사항 구체화 결과 검토

AI가 입력한 요구사항을 Epic · Feature · UserStory · 사용자 흐름으로 구체화합니다. 내용이 모호하면 선택형 질문으로 확인합니다. 검토 결과 수정이 필요하면 의견을 적어 다시 생성하거나 직접 수정할 수 있습니다.

4-6단계 · 화면 확인 · 구현 · 병합

4단계 · 화면 초안 검토

요구사항이 구체화되면 **스토리보드를 자동으로 생성**합니다. 화면을 사용 순서에 따라 배치하므로, 관계자는 주요 기능과 화면 이동 흐름을 보며 초안을 검토할 수 있습니다.

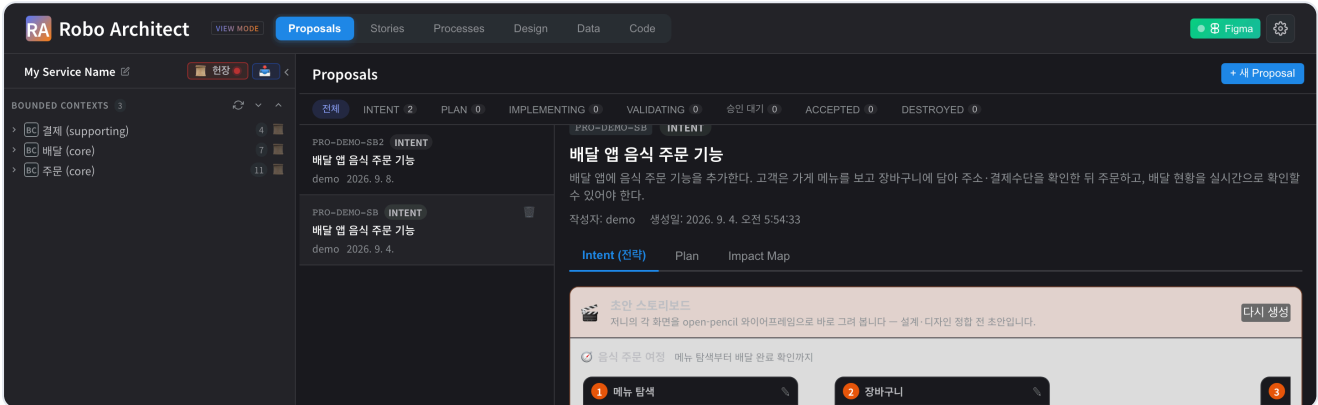


그림 9. 제안 상세 — 상단 탭(Intent · Plan · Impact Map)과 초안 스토리보드. 화면 카드를 클릭하면 편집기가 열립니다. *데모 데이터를 사용한 화면입니다.*

5단계 · 구현 계획 확정 및 코드 작성

Plan

확정된 요구사항과 프로젝트 설계 원칙에 따라 상세 설계와 배포 · 연동 · 저장소 구성 계획을 작성합니다.

Implement

별도 작업 공간에서 Claude Code가 구현합니다. Code 탭에서 진행 상황을 확인하고 작업을 중단하거나 수정 지시를 전달할 수 있습니다.

6단계 · 검증 및 최종 병합

인수 조건(GWT)을 기준으로 결과를 검증한 뒤 승인하면, **코드 병합과 그래프 반영이 함께** 일어납니다. 갱신된 그래프는 최신 설계를 담고, 다음 변경의 기준이 됩니다.

검증

인수 조건 충족 여부와 구조 검사 결과를 확인합니다.

병합

코드 병합과 설계 반영을 함께 처리하며, 실패하면 양쪽을 되돌립니다.

이력

승인자, 승인 시점, 변경 내용을 이력으로 기록합니다.

중단한 작업을 이어서 진행할 수 있습니다. 문서 분석, 요구사항 구체화, 구현 과정의 중간 상태를 저장합니다. 다시 접속하거나 새로고침한 뒤에도 기존 작업에 연결할 수 있습니다.

보안 · 거버넌스

AI가 생성한 설계와 코드를 업무에 적용하려면 데이터 처리 범위와 변경 승인 절차를 관리해야 합니다. Robo Architect는 **고객사 환경에 설치**하며, 조직의 정책에 맞춰 LLM 연결과 변경 통제를 구성할 수 있습니다.

데이터 주권

- **고객사 환경에 데이터를 저장합니다.** 사내 서버 또는 담당자 PC에 설치해 설계 그래프와 소스 코드를 관리합니다.
- **LLM 연결 대상을 지정할 수 있습니다.** 사내 게이트웨이나 자체 운영 모델(OpenAI 호환)에 연결하면 프롬프트를 외부로 전송하지 않는 구성이 가능합니다.
- **설정에서 모델을 변경할 수 있습니다.** 사내 정책과 사용 목적에 맞게 LLM 공급자와 모델을 선택합니다.

변경 통제

항목	내용
승인 게이트	사용자의 명시적 승인 후 대상 브랜치와 설계 그래프에 변경 사항을 최종 반영합니다
자기 승인 금지	변경을 제출한 사람이 그 변경을 승인할 수 없습니다
동시 편집 보호	버전 불일치 시 덮어쓰지 않고 충돌로 알립니다
코드 격리	구현은 전용 브랜치의 격리된 작업 공간에서만 수행하며, 원본 브랜치는 병합 전까지 변경되지 않습니다
되돌리기	승인을 취소하면 설계 반영을 되돌리며, 코드 되돌림 여부도 선택할 수 있습니다
감사 로그	요청별 추적 번호와 구조화된 로그를 기록하고, 변경 상태 이력을 그래프에 저장합니다

조직의 설계 기준을 작업 절차에 반영

요구사항 구체화부터 계획, 구현, 검증까지의 절차를 스킴 파일로 관리합니다. 명명 규칙, 필수 산출물, 금지 패턴 등 **조직의 설계 기준을 각 단계에 반영**할 수 있습니다. 기술 스택과 아키텍처, 저장소 구성은 프로젝트 설계 원칙으로 저장해 이후 계획에도 적용합니다.

도입 시 고객사의 보안 정책에 맞춰 구성을 확정합니다. 망 분리 수준, 계정 연동 방식, 로그 수집 범위를 함께 검토하고 필요한 설정을 협의합니다.

설치 및 납품 방식

데스크톱 설치 프로그램으로 필요한 실행 환경을 구성할 수 있습니다. Python, Node.js, 데이터베이스를 각각 설치할 필요가 없으며, 고객사 환경에 따라 서버 설치 방식도 선택할 수 있습니다.

구분	절차	소요
데스크톱	설치 프로그램 실행 → 컨테이너 런타임 · 그래프 DB · 백엔드까지 자동 실행	수십 분
서버 설치	Compose로 백엔드 · 그래프 DB · 렌더 서비스 실행 후 사내 도메인 연결	수십 분
폐쇄망	컨테이너 이미지를 파일로 반입 → 검증 → 로드 → 실행	반입 절차 별도
업그레이드	새 버전으로 교체 후 다시 실행. 그래프 스키마는 실행 시 자동 적용	분 단위

데스크톱 설치 구성

단일 설치 프로그램

Windows(NSIS), macOS(DMG), Linux(ApplImage) 용 설치 파일을 제공합니다. 앱과 함께 백엔드를 실행하고 종료하며, 자동 업데이트를 지원합니다.

rootless 컨테이너

Podman을 사용해 데몬 없이, 관리자 권한 없이 구동합니다. 상용 라이선스가 필요한 데스크톱 컨테이너 제품에 의존하지 않습니다.

선택 설치 팩

레거시·데이터 분석 팩은 설치 시 선택하거나 나중에 추가·제거할 수 있습니다.

외부 DB 연결

그래프 데이터베이스를 별도로 설치하는 대신, 사내에서 운영 중인 데이터베이스에 연결할 수도 있습니다.

권장 실행 환경

담당자 PC에는 메모리 16GB 이상과 컨테이너 런타임이 필요하며, 사내 게이트웨이 또는 외부 API를 통해 LLM에 연결할 수 있어야 합니다. 서버 사양은 동시 사용자 수에 맞춰 별도로 산정합니다.

산출물은 고객 자산으로 납품됩니다. 설계 그래프는 표준 Cypher로 내보낼 수 있고 생성된 코드는 고객 저장소에 그대로 남으므로, 제품 사용을 종료한 뒤에도 설계 데이터와 코드를 보유하고 활용할 수 있습니다.

제품 개발 및 품질 검증 체계

Robo Architect 개발팀은 **명세 작성부터 구현과 검증까지 일관된 절차**를 적용합니다. 설계 결정의 근거를 기록하고, 자동 테스트와 정적 분석으로 구현 결과를 점검합니다.

명세를 먼저 작성하는 개발 절차

모든 기능은 명세 작성, 구현 계획 수립, 작업 목록 작성, 구현 순서로 개발합니다. 산출물은 저장소에 함께 보관하며, 현재 **47개의 기능 명세**에 요구사항과 주요 설계 결정의 근거가 기록되어 있습니다.

제품 개발의 10가지 원칙

원칙	내용
I. 설계 정보의 기준	설계 정보를 그래프에서 관리하고 별도의 기준 데이터를 두지 않습니다
II. 도메인 용어	명령과 이벤트에 업무 의미를 나타내는 도메인 용어를 사용합니다
III. 진행 상황 표시	수 초 이상 걸리는 작업은 진행 상황을 실시간으로 표시합니다
IV. 사용자 승인	계획과 적용을 분리하고 승인 후 변경 사항을 반영합니다
V. 기능별 모듈	백엔드와 프론트엔드를 기능별로 대응시키고 모듈 간 직접 참조를 제한합니다
VI. 공급자 선택	LLM 공급자와 모델을 코드 수정 없이 설정으로 변경합니다
VII. 처리 이력 추적	요청별 추적 번호와 구조화된 로그를 기록합니다
VIII. 화면 생성	화면은 지정된 렌더링 엔진을 통해 생성합니다
IX. 플러그인 연동	Figma 플러그인과 백엔드의 개발 · 검증 절차를 함께 관리합니다
X. 스킴 기반 절차	방법론과 작업 절차를 제품 코드와 분리해 스킴 파일로 관리합니다

자동 검증

49

단위 테스트 모듈
기능별 pytest

70

E2E 시나리오
Playwright — 실제 브라우저

정적 분석

전체 코드
ruff · 타입 검사

본 소개서에는 실제 제품 실행 화면을 수록했습니다. 모든 화면은 상시 가동 중인 환경에서 캡처했습니다.

기반 제품 MSAEZ의 도입 실적

Robo Architect는 이벤트스토밍 기반 설계 · 코드 생성 도구인 **MSAEZ(MSA Easy)**의 개발 및 운영 경험을 바탕으로 확장한 제품입니다. 다음은 MSAEZ의 산업 현장, 공공 플랫폼, 교육 분야 활용 사례입니다.

대기업 실무

포스코DX

애플리케이션 · 마이크로서비스 설계와 소스 코드 생성 도구로 사용 중

공공 플랫폼

DPG 통합테스트베드

첨단 디지털 개발지원 도구(SaaS)로 등록·운영

9,655명

MSA School 누적 수강생

누적 교육 325회 · 17,864시간, 표준 실습 도구로 활용

① 산업 현장 — 포스코DX

MSAEZ는 포스코DX에서 **애플리케이션과 마이크로서비스의 설계 및 소스 코드 생성**에 사용되고 있습니다. 이벤트스토밍으로 도출한 설계 모델을 바탕으로 코드를 생성하는 데 활용됐습니다.

② 공공 — DPG 통합테스트베드

DPG(디지털플랫폼정부) 통합테스트베드는 한국지능정보사회진흥원(NIA)이 운영하는 플랫폼입니다. 중소기업, 스타트업, 시민 개발자, 공공기관이 민간 · 공공 데이터와 클라우드 자원을 활용해 **서비스를 개발 · 시험 · 검증**할 수 있도록 지원합니다. 민간의 SaaS를 개발 지원 도구로 제공합니다.

MSA Easy(MSAEZ)는 이 테스트베드에 등록되어 **마이크로서비스의 분석 · 설계 · 구현 · 배포 전 과정을 자동화**하는 SaaS로 제공되고 있습니다.

③ 교육 — MSA School

MSAEZ는 클라우드 네이티브 개발 교육기관 **MSA School의 표준 실습 도구**로 사용됩니다. 수강생은 이벤트스토밍으로 도메인을 설계하고, 모델에서 코드를 생성해 배포하는 과정을 실습합니다.

누적 교육 실적은 **325회, 수강생 9,655명, 17,864시간**입니다. 교육은 기초 · 중급 · 고급의 3단계로 구성되며, Biz(이벤트스토밍 · DDD), Dev(Spring Boot · Kafka), Ops(Kubernetes · GitOps)를 다룹니다.

초기 설계에서 지속적인 변경 관리로 확장했습니다. Robo Architect는 MSAEZ의 설계 · 코드 생성 기능을 바탕으로 변경 영향 분석, 제안 단계별 관리, 화면 초안 생성, 코딩 에이전트 연동, 폐쇄망 설치를 지원합니다.

* 교육 수치는 MSA School 공개 집계 기준입니다. 구축 범위와 도입 효과는 영업 담당자가 개별 안내합니다.

도입 절차 및 일정

환경 검토와 PoC(개념 검증)를 거쳐 본 도입까지 **4단계로 진행**합니다. 아래 일정은 표준 예시이며, 고객사의 환경과 검토 절차에 따라 조정합니다.



PoC 주요 검증 항목

- 실제 요구사항 문서의 주요 개념과 관계가 **도메인 그래프에 적절히 반영**되는지
- 실제 변경 사례에 대해 **관련 설계 요소와 소스 파일**을 식별하는지
- 생성된 화면 초안으로 **이해관계자가 기능과 사용 흐름**을 이해할 수 있는지
- 기존 저장소 구성과 브랜치 운영 방식에 맞춰 **구현 · 병합**을 진행할 수 있는지
- 폐쇄망 설치와 사내 LLM 연동이 **고객사의 보안 정책**을 준수하는지

도입 시 협의할 사항

LLM 실행 경로

외부 API, 사내 게이트웨이, 자체 운영 모델 중 연결 방식을 정합니다.

대상 저장소 · 브랜치 전략

작업할 저장소와 최종 병합 대상 브랜치를 정합니다.

조직 공통 설계 기준

기술 스택, 아키텍처, 저장소 구성의 공통 기준을 정합니다.

스킬 커스터마이징

조직의 설계 절차와 산출물 양식을 스킬에 반영합니다.

30분 제품 시연 및 PoC 환경 협의

요구사항 입력부터 설계 그래프와 화면 생성까지 실제 제품으로 시연합니다. 고객사의 업무 문서 일부를 미리 제공해 주시면 **해당 자료를 활용한 시연**을 준비합니다.



Robo Architect

AI ARCHITECTURE WORKBENCH

도입 문의

요구사항부터 구현까지, 변경을 함께 관리하세요.

도입 상담, 제품 시연, 견적을 문의해 주세요. 담당 전문가가 영업일 기준 1일 이내에
답변드립니다. 필요하시면 제품 시연 일정도 함께 안내합니다.

회사

(주)유엔진솔루션즈

제품

Robo Architect

문서

Rev. 2026.09

한 흐름으로 이어지는 네 단계

요구사항

문서 · 대화에서 그래프로

설계

DDD · 이벤트스토밍

화면

초안부터 Figma까지

코드

격리 환경 구현 · 병합

(주)유엔진솔루션즈 · uEngine Solutions, Inc.

© 2026 uEngine Solutions. 본 문서 Rev. 2026.09 · 기재된 수치 · 화면은 작성 시점 기준이며 사전 통지 없이 변경될 수 있습니다.

uEngine